UNIT 2
Computational Thinking & Algorithms

11TH COMPUTER SCIENCE SINDH TEXTBOOK BOARD

AWINASH GOSWAMI

https://awinashgoswami.github.io

Agenda

- Computational Thinking
- Algorithm
- Decomposition
- Pattern Recognition
- Abstraction

Computational Thinking

- Computational Thinking is a systematic approach to solving a problem using computer. The systematic approach include the properties algorithm, decomposition, pattern recognition, abstraction.

Computational Thinking Properties

- **Algorithms –** Step-by-step solutions to a problem. Each step defines a rule to follow. Essential for automation and efficiency.
  - ***Example**: **Withdrawing Money from ATM -** Insert card. Enter PIN. Select "Withdraw." Enter amount. Collect cash. Take card back.*

- **Importance of Algorithm:**
  - Algorithm help in selecting solutions that optimize time and memory usage, which is crucial for large-scale application.
  - Once developed, an algorithm can be reused for similar problems, saving both time and effort.
  - Algorithm ensure accuracy and consistency, producing correct results when followed properly and reducing the likelihood of human error.

- **Pseudocode**: a human-readable representation of an algorithm using plain language and structure programming keywords.

Computational Thinking Properties Cont....

- **Decomposition** – Breaking large problems into smaller, manageable sub-problems and solving them one by one.
  - ***Example:*** *Instead of thinking about the whole wedding at once, break it into smaller tasks (Book a hall, Order food, Send invitations, Arrange decorations, Hire a photographer).*

- **Pattern Recognition** – Identifying similarities and differences in problems or data to create a general solution.
  - ***Example: Weather in Sindh -*** *Every year, summer is hot and dry, and monsoon brings rain, by recognizing this yearly pattern, farmers decide when to plant crops.*

Computational Thinking Properties Cont....

- **Abstraction** – Focusing on important information and ignoring irrelevant details. Reduces complexity and makes solutions easier to develop.

  - ***Example: Using Google Maps -*** *When you use Google Maps to go from Karachi to Hyderabad, you only see* ***roads and landmarks, y****ou don't see trees, shops, or small houses — those details are* ***hidden*** *because they are not important for navigation.*

  - ***Example 2: Using a Mobile Phone -*** *You tap on the* ***WhatsApp icon*** *to open the app. You don't need to know the* ***complex code*** *behind it, the icon is an* ***abstraction*** *of all that code.*

Sorting Algorithm

- Sorting is the process of arranging items in a collection of data.
- It may be ascending – lowest to highest (0-9, A-Z) or descending highest to lowest (9-0, Z-A).
- Sorting organizes data which makes data more accessible for searching, retravel and analysis.
- In this unit scope we will study sorting algorithms:
  - Bubble Sort
  - Selection Sort

Bubble Sort

- **Bubble sort** is a simple sorting algorithm that repeatedly steps through a list, compares adjacent items, and swaps them if they're in the wrong order.
- The advantage of bubble sort is that it is the easiest sorting algorithm to understand and explain; it doesn't create a list or use extra RAM to perform sorting.
- The disadvantages for bubble sort are;
  - It uses nested loop; the time it takes to sort grows exponentially.
  - Every swap requires the computer to perform three distinct operations.
  - It is okay for smaller size of array like 10 to 20 items but real-world Big Data it is useless.

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[] = {5, 1, 4, 2, 8};
    int n = sizeof(arr) / sizeof(arr[0]);

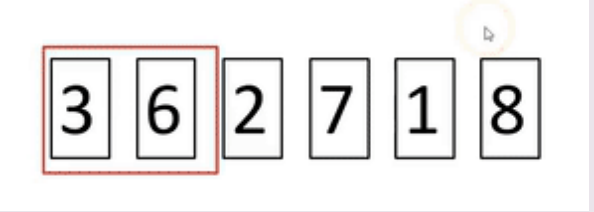
    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;

        // Last i elements are already in place
        for (int j = 0; j < n - 1 - i; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
                swapped = true;
            }
        }

        // If no swaps happened, the array is already sorted
        if (!swapped) break;
    }

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```



Selection Sort

- **Selection sort** is a simple sorting algorithm that repeatedly finds the smallest item in the unsorted part of the list and moves it to the front.
- **The advantages of selection sort are that** it is very easy to understand. In every pass, it performs at most one swap which makes sorting quite efficient. It does not need extra memory.
- The disadvantages of Selection Sorts are it has also nested loops. As the list grows, the time it takes to finish grows very fast. It is same like bubble sort, it performs well on smaller size of data but for larger size of data it is inefficient.

```cpp
#include <iostream>
using namespace std;
int main() {
    int arr[] = {5, 1, 4, 2, 8};
    int n = sizeof(arr) / sizeof(arr[0]);

    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;  // assume the first unsorted item is the smallest

        // Find the smallest item in the remaining unsorted part
        for (int j = i + 1; j < n; j++) {
            if (arr[j] < arr[minIndex]) {
                minIndex = j;
            }
        }

        // Swap the smallest item into position i
        if (minIndex != i) {
            int temp = arr[i];
            arr[i] = arr[minIndex];
            arr[minIndex] = temp;
        }
    }

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```



Searching Algorithm

- Searching is the process of finding an item in a collection of data.
- These algorithm help to find the position, existence, or value of the target element.
- In this unit scope we will study searching algorithms:
  - Linear Search
  - Binary Search

Linear Search

- **Linear search** (also called sequential search) is the simplest way to find an item in a list: you check each item one by one, from start to finish, until you find what you're looking for or reach the end.
- **Advantages:**
  Linear Search works on any list, whether the data is sorted or not. The code is just a simple loop and an if statement, so it's very easy to write. For small lists (like 10 or 20 items), it's actually faster than other algorithms because there is no setup time. It also needs no extra memory to work.
- **Disadvantages:**
  It gets very slow on big lists. If a list has 1 million items and your target is at the end, it has to check all 1 million items, and the time grows linearly as the list gets bigger. Also, if you have to search the same list over and over, it's much smarter to sort it once and use a faster search like Binary Search.

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[] = {5, 1, 4, 2, 8};
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 4;
    int foundIndex = -1; // -1 means "not found"

    // Loop through the array one by one
    for (int i = 0; i < n; i++) {
        if (arr[i] == target) {
            foundIndex = i;
            break; // Stop searching once we find it
        }
    }

    // Check the result and print
    if (foundIndex != -1) {
        cout << "Target found at index: " << foundIndex << endl;
    } else {
        cout << "Target not found in the array." << endl;
    }

    return 0;
}
```



Binary Search

- Binary Search works by repeatedly dividing the search interval in half and comparing the middle element to the target element. If the target is smaller, it searches the left half; if larger, it searches the right half. It repeats this halving process until the item is found or the list is empty. Its only limitation is that the list must be sorted.
- **Advantages:**
  Binary search is incredibly fast because it halves the search area at every step, finding an item in a million-item list in just 20 steps. This makes it perfect for handling large amounts of data. It is especially great for static lists, like phone books, that you need to search many times, saving a massive amount of time.
- **Disadvantages:**
  The biggest catch is that binary search only works on lists that are already sorted; it fails completely on scrambled data. If a list isn't sorted, spending time to sort it first might be slower than just doing a simple linear search. Also, it only works well with arrays (where you can jump to the middle) and is more prone to coding errors, like infinite loops, than linear search.

```cpp
#include <iostream>
using namespace std;

int main() {
    int arr[] = {1, 2, 4, 5, 8, 9, 11}; // List must be sorted!
    int n = sizeof(arr) / sizeof(arr[0]);
    int target = 5;

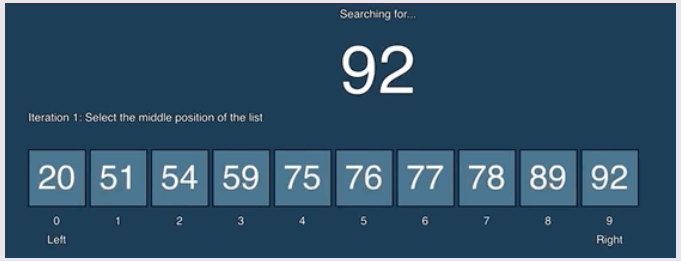
    int low = 0;
    int high = n - 1;
    int foundIndex = -1; // -1 means not found

    while (low <= high) {
        int mid = low + (high - low) / 2; // Find the middle index

        if (arr[mid] == target) {
            foundIndex = mid; // Found it!
            break;
        }
        else if (arr[mid] < target) {
            low = mid + 1; // Target is larger, ignore left half
        }
        else {
            high = mid - 1; // Target is smaller, ignore right half
        }
    }

    if (foundIndex != -1) {
        cout << "Target " << target << " found at index: " << foundIndex << endl;
    } else {
        cout << "Target " << target << " not found in the array." << endl;
    }

    return 0;
}
```



Evaluation Criteria of Algorithm

Correctness

An algorithm is correct if it produces the expected output. To be correct, it must:

- Solve the exact problem as defined.
- Handle all inputs gracefully, including edge cases like negative numbers, empty strings, or null values.
- Always finish, reaching a logical end without getting stuck in an infinite loop.
- (Example: Reading a number and checking if the remainder of dividing by 2 is 0 to print "Even" or "Odd".)

Efficiency

We measure any algorithm efficiency through time and space as resources. In simple words we find how fast it runs as input size grows and how much memory it uses. For this we check.

- Does the algorithm scale well with larger input sizes?
- Is there a way to make it faster or use less memory?
- Is the algorithm optimized for real-world use?
- **Example:** A linear search is less efficient than binary search when dealing with large sorted data sets.

Clarity

Algorithm must be easy to read, understandable and easy to modify by other users. For clarity the algorithm should follow the following principles.

- Steps are clearly written.
- Variable names are meaningful.
- The logic structure is coupled with indentation and comments.
- The flow of control (IF, LOOP, etc.) are easy to follow.

How to Decide Which Algorithm to Use

- It can be difficult to decide which algorithm to use. Before deciding which algorithm is best for our problem, we must consider the following questions:
  - What is the type of problem?
  - What is input size?
  - Is the data sorted?
  - Do we need Speed and Memory efficiency?

How to Decide Which Algorithm to Use

- **Simple Decision Guide:** Now we have some question and also have some problem to decide what the best algorithm for specific problem is.
    - If searching on unsorted data then use Linear Search algorithm.
    - If searching on sorted data then use Binary Search algorithm.
    - If data needs ordering, then use Sorting algorithm.
    - If problem repeats many times, then use Dynamic Programming algorithm.

Example:

- Find a number in a sorted list
- Is the data sorted?
- Is a fast result needed?

- **Choose:** Binary Search